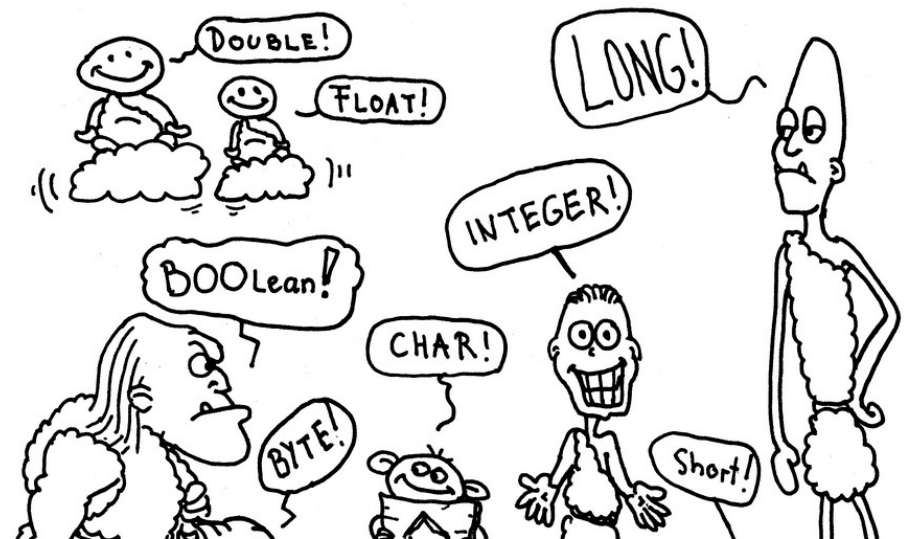


Datentypen

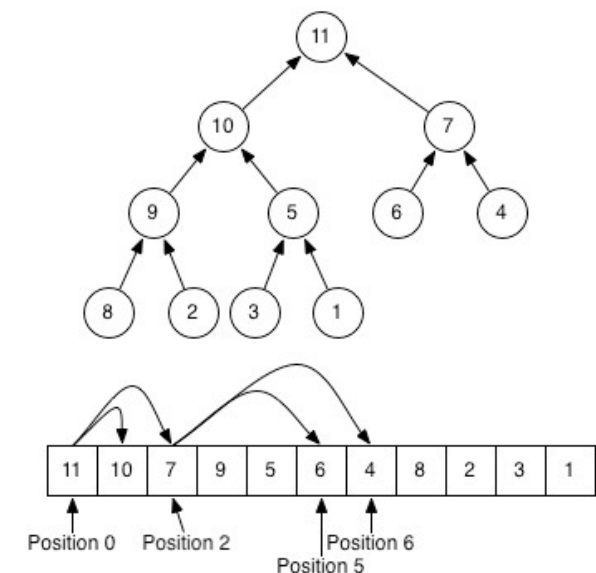
Definition Datentyp:

- Legt die Eigenschaften einer Variablen fest und bestimmt die Werte oder Wertemengen die die Variable annehmen kann.
- Anzahl und Arten von zulässigen Operationen



Definition Datenstruktur:

- Beschreibung der Eigenschaften der Daten und der zur Verfügung stehenden Zugriffsoperationen dieser Daten.
- Menge+Operationen
- Implementierung kann verschieden sein



Beispiel: Zahlen vom Datentyp Integer

Operationen auf dem Datentyp: Addition, Multiplikation, Division

Bei Division muss Ergebnis abgeschnitten werden:

$$9/3 = 3$$

$$10/3 = 3$$

-> Operationen auf dem Datentyp müssen sinnvoll deklariert sein

Einfache Datentypen: Standardtypen

- int
 - unsigned int, signed int, long int
- float
 - double, long double
- char
 - (-128..127)

Einfache Datentypen:

- Aufzähltyp:
 - enum figur {Rechteck,Quadrat,Ellipse,Kreis}
 - enum wochentag {Montag,Dienstag, Mittwoch, Donnerstag,Freitag,Samstag, Sonntag}
- Zeigertyp
 - * Dereferenzierungsoperator
 - & Adressoperator

Datenstruktur Array

- Zusammenfassung von Komponenten desselben Datentyps
- Zugriff über ganzzahligen Index
- konstante oder variable Dimensionierung

Datenstruktur Record

- fassen Komponenten beliebiger Datentypen zusammen
- Zugriff auf Inhalt über Punkt oder Pfeiloperator
- Auch Arrays von records möglich

Datenstrukturen

Beispiel: Visitenkarte

Menge verschiedener Daten (Namen einer Person, Titel, Firma usw.)

Die Adresse gliedert sich wiederum in Strasse, Postleitzahl, Wohnort usw. auf

Jedes dieser Merkmale muss einem Datenobjekt zugeordnet werden und einen entsprechenden Datentyp bekommen

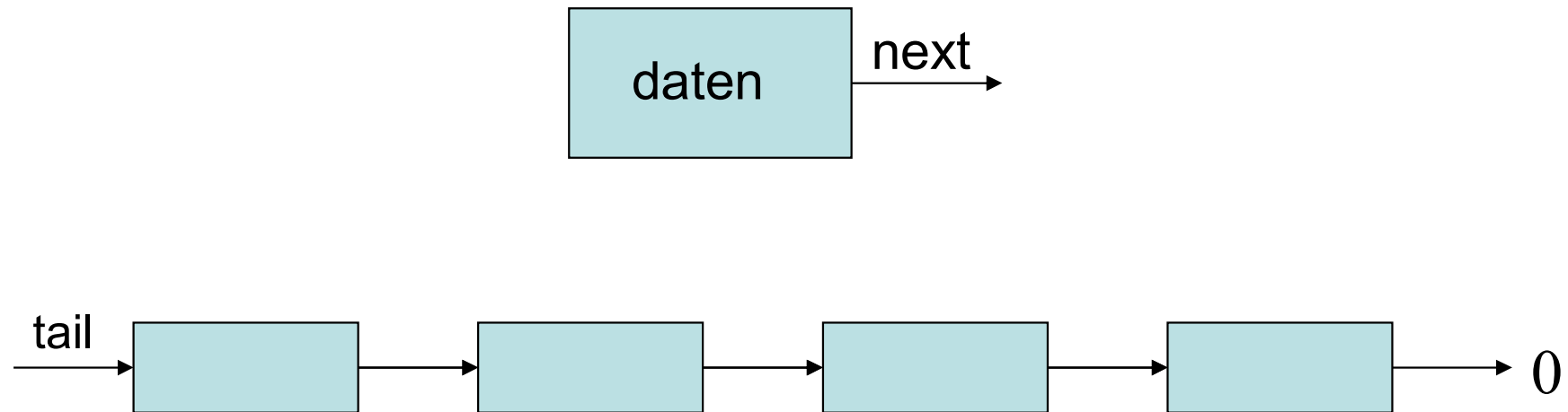
Alle Merkmale zusammen nennt man Datensatz oder Record

Dynamische Datenstrukturen

- Arrays und Records, die durch Zeiger verbunden sind
- Anlegen und löschen eines neuen Objektes vom Typ t durch:
 new t;
 delete t;
- Zugriff auf Inhalt nur über * und -> Operator möglich

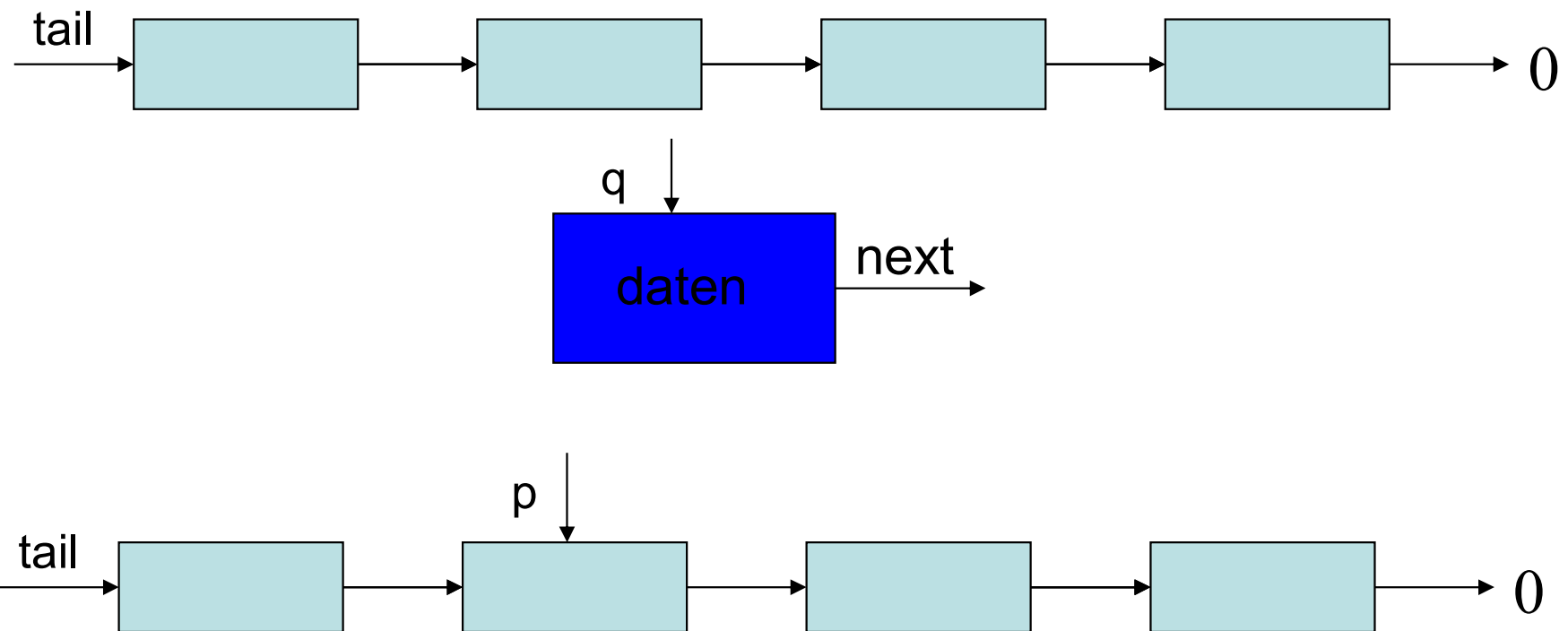
Dynamische Datenstrukturen

- Verbindung der Knoten durch Zeiger
- einfach verkettete Liste
 - jeder Knoten enthält einen Zeiger auf einen anderen Knoten
 - letzter Knoten zeigt auf 0



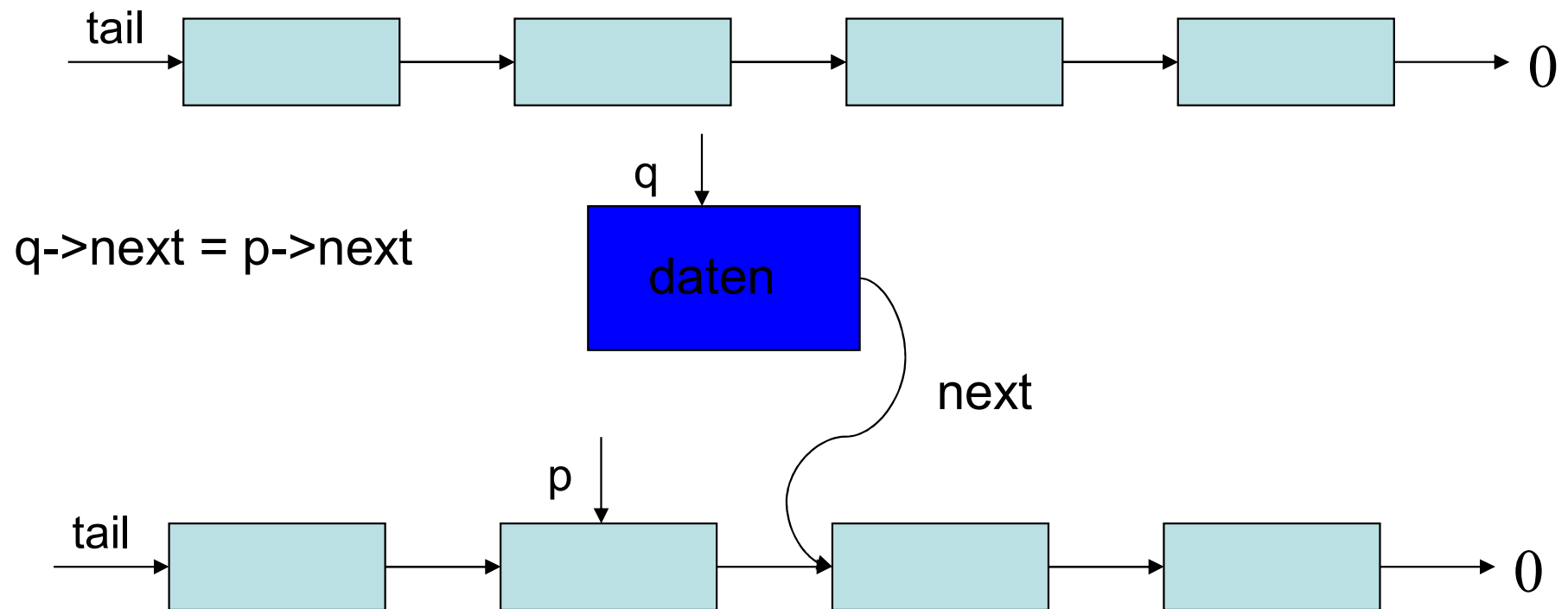
Dynamische Datenstrukturen

Einfüge- und Löschoperation in verketteter Liste:
nur Zeigeroperationen notwendig. Beispiel Einfügen nach p:



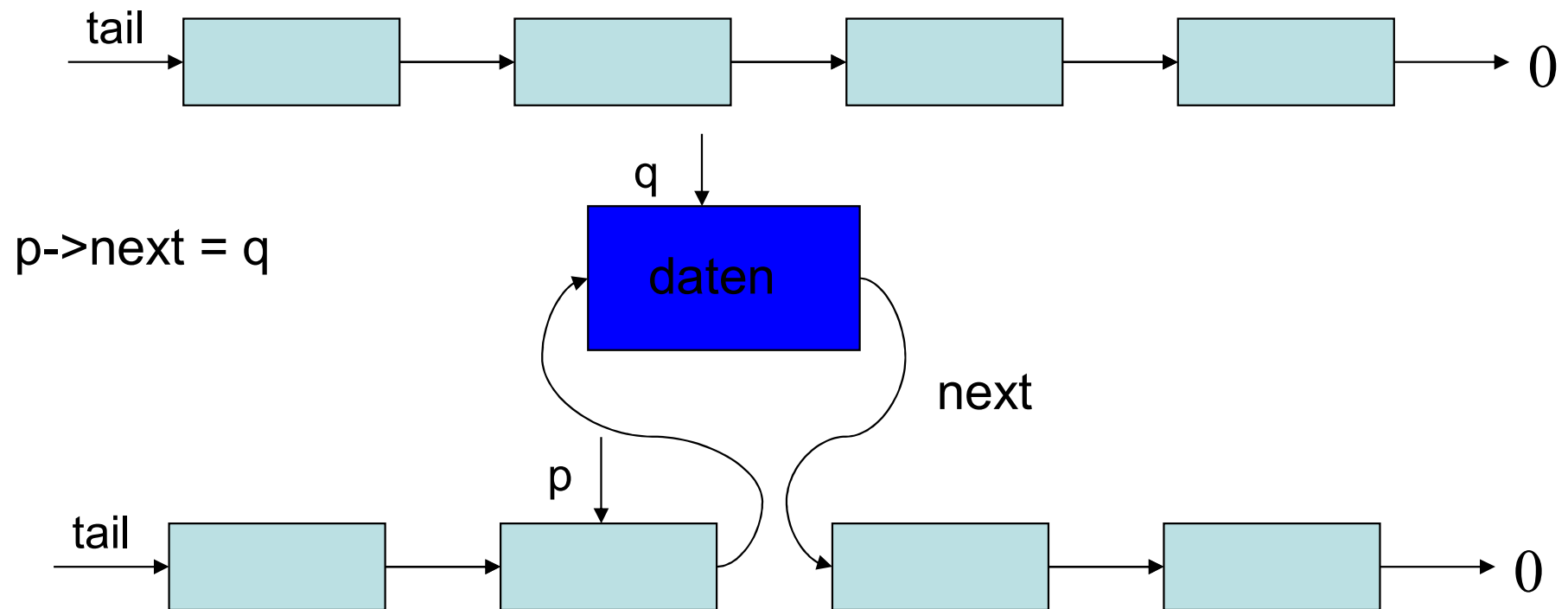
Dynamische Datenstrukturen

Einfüge- und Löschoption in verketteter Liste:
nur Zeigeroperationen notwendig



Dynamische Datenstrukturen

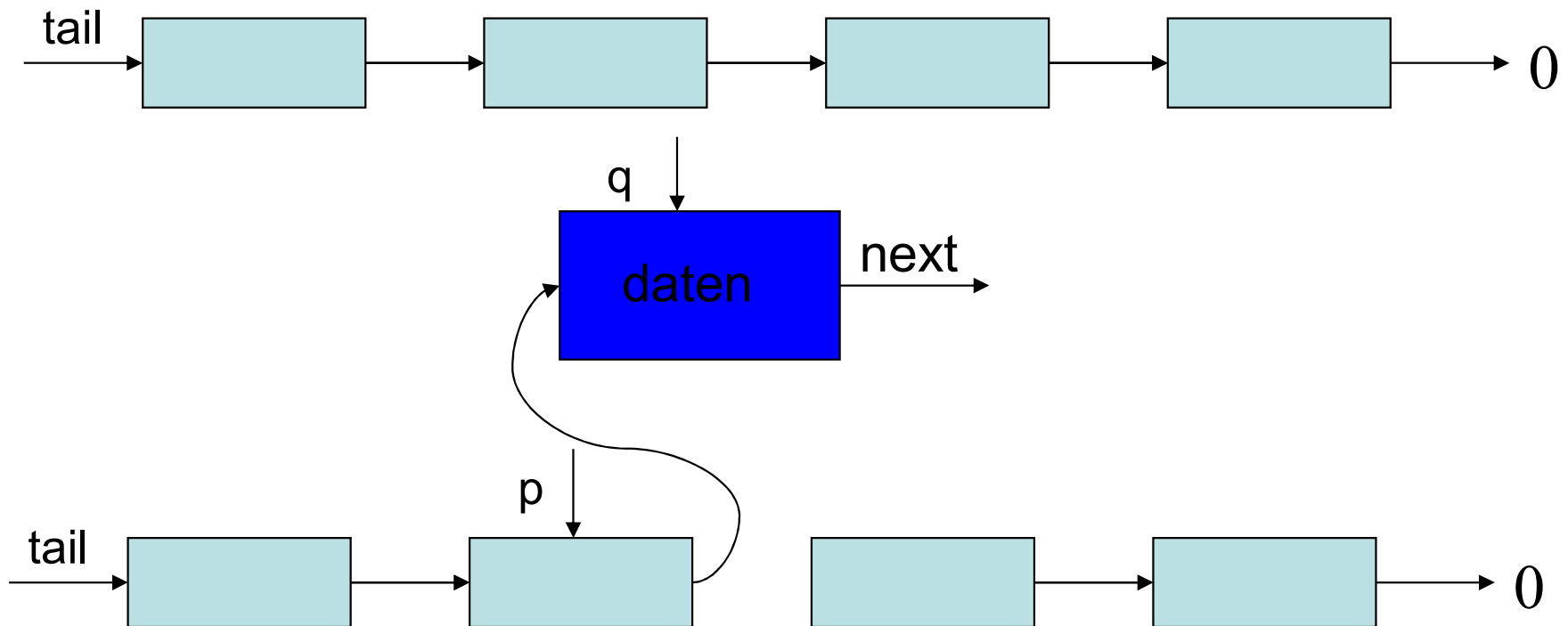
Einfüge- und Löschoperation in verketteter Liste:
nur Zeigeroperationen notwendig



Dynamische Datenstrukturen

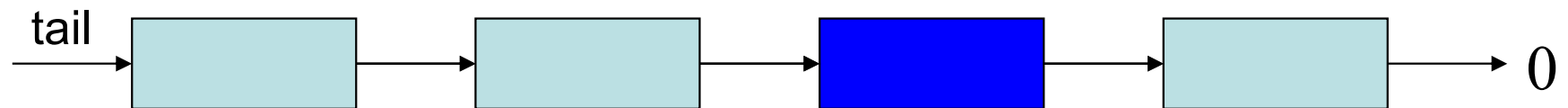
Gefahr des „Abhängens von Knoten“

$p \rightarrow \text{next} = q$

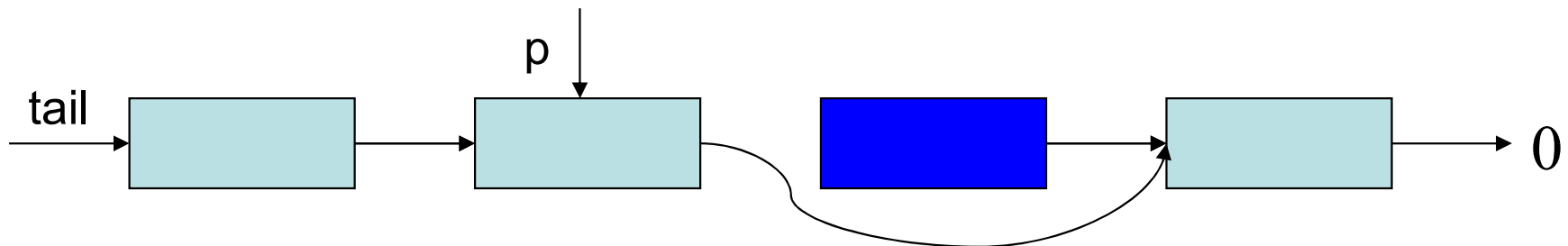


Dynamische Datenstrukturen

Gefahr der „Speicherfresser“ beim Löschen:

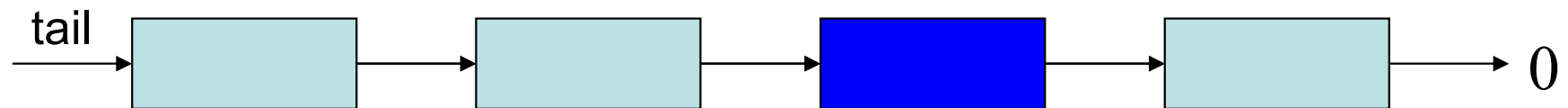


$p \rightarrow \text{next} = p \rightarrow \text{next} \rightarrow \text{next}$

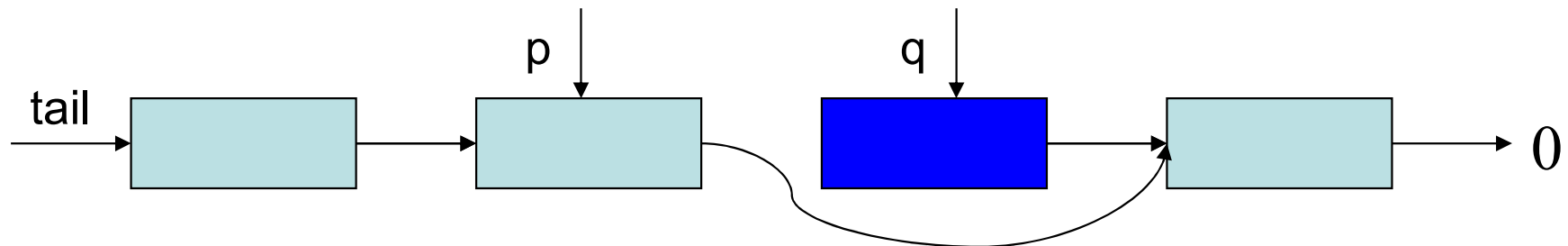


Dynamische Datenstrukturen

Gefahr der „Speicherfresser“ beim Löschen:

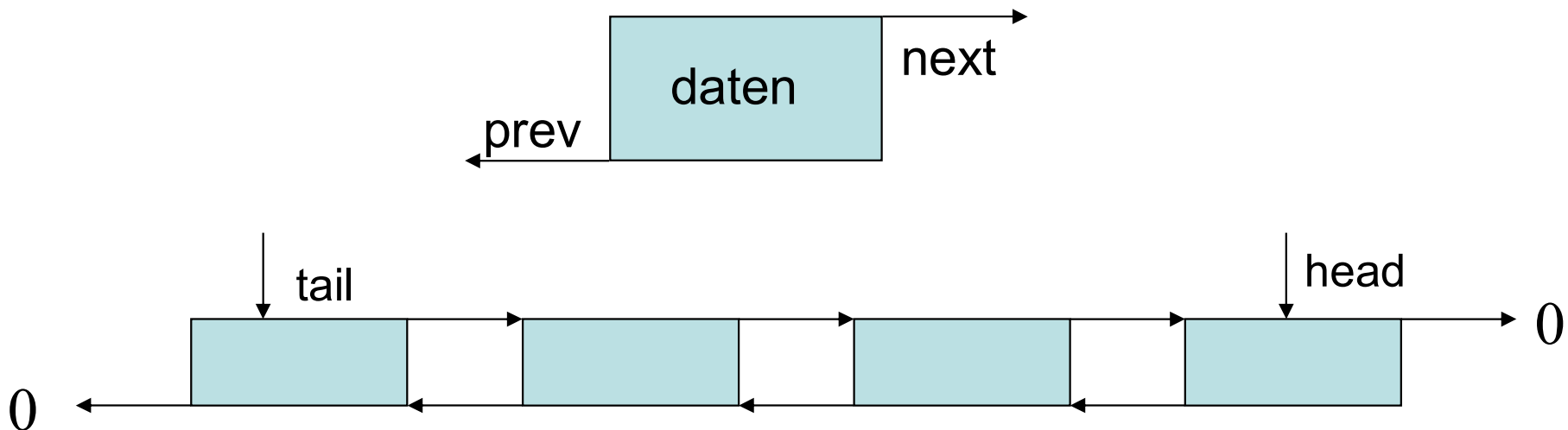


```
q=p->next;  
p->next = q->next;  
delete q;
```



Dynamische Datenstrukturen

- doppelt verkettete Liste
 - jeder Knoten enthält 2 Zeiger:
 - next: nächster Knoten
 - prev: vorheriger Knoten
 - Zugriff über Anfangs- (head) oder Endknoten (tail)



Abstrakte Datentypen:

- Abstrakter Datentyp = Datentypen + Funktionen
 - Daten werden gekapselt (private)
 - Zugriff auf die verborgenen Daten über Operationen (öffentliche Schnittstelle)
 - Implementierung ist nach außen hin verborgen und austauschbar

ADT Stack:

- Liste mit eingeschränkten Operationen
 - es kann nur am Anfang eingefügt werden
 - Last-In-First-Out = LIFO

Datenstruktur Stack:

Benötigte Operationen in der Datenstruktur Stack:

s.top - liefert das oberste Element

s.pop - entfernt das oberste Element

s.push(x) – legt ein Element auf den Stack

Für den Benutzer der Datenstruktur sichtbare Operationen:

```
template <class Tr>
class Stack    // Stapel für Records von beliebigem Typ Tr
{
public:
    Stack(unsigned max); // leerer Stapel für max Records
    ~Stack();           // Destruktor
    void Push(Tr r);    // legt Record r oben auf Stapel
    Tr Pop();           // holt obersten Record vom Stapel
    Tr Top();           // liefert den Wert des obersten Records
    unsigned Length();  // Anzahl der Records im Stapel
    boolean IsEmpty();  // true, wenn Stapel leer, sonst false
    boolean IsFull();   // true, wenn Stapel voll, sonst false
```

Für den Benutzer der Datenstruktur unsichtbare Daten:

```
private:
    unsigned m;        // Arraylänge m
    unsigned top;      // oberste array-Position im Stack
    Tr *a;             // Zeiger auf Array
};
```

Implementierung der Datenstruktur Stack als Array:

```
// Konstruktor: legt Speicherplatz für max Records an
// Setzt top auf 0 und m auf max
template <class Tr>
Stack<Tr>::Stack(unsigned max) {
    m=max;
    a=new Tr[m];
    top=0;
}
// Destruktor loescht alle Eintraege im Stack
template <class Tr>
Stack<Tr>::~~Stack() {
    delete a[];
}
```

Implementierung der Datenstruktur Stack als Array:

```
// oben im Stapel ablegen, Bedingung: Stapel nicht voll
template <class Tr>
void Stack<Tr>::Push(Tr r)
{
    if (!IsFull())
    {
        a[top]=r;
        top++;
    }
    else
        printf(„Stack ist voll - Push() nicht möglich“);
}
```

Implementierung der Datenstruktur Stack als Array:

```
// obersten Record holen und entfernen,  
// Bedingung: Stapel nicht leer  
template <class Tr>  
Tr Stack<Tr>::Pop()  
{  
    Tr r;  
    if (!IsEmpty())  
    {  
        top--;  
        r=a[top];  
        return r;  
    }  
    else  
        printf(„Stack ist leer - Pop nicht möglich“);  
}
```

Implementierung der Datenstruktur Stack als Array:

```
// Wert des obersten Records im nicht leeren Stapel zurückgeben
template <class Tr>
Tr Stack<Tr>::Top()
{
    Tr r;
    if (!IsEmpty())
    {
        r=a[top-1];
        return r;
    }
    else
        printf(„Stack ist leer - Top() nicht möglich“);
}
```

Implementierung der Datenstruktur Stack als Array:

```
// Anzahl der Records im Stapel zurückgeben
template <class Tr>
unsigned Stack<Tr>::Length()
{
    return top;
}
```

Implementierung der Datenstruktur Stack als Array:

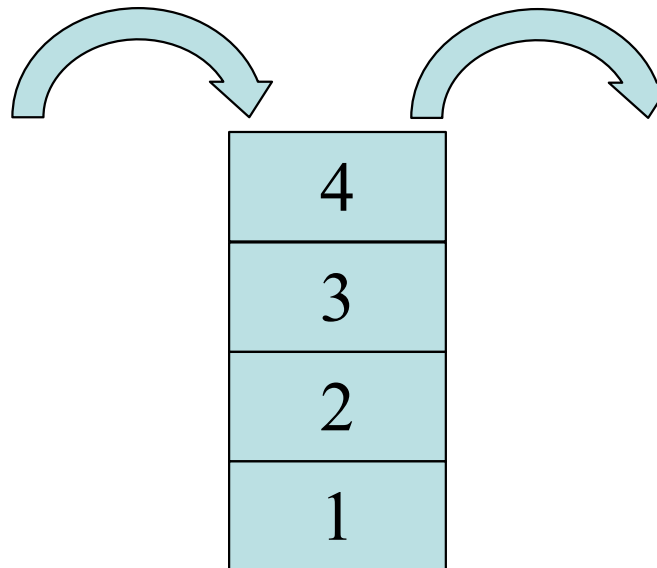
```
// prüfen, ob Stapel leer ist
template <class Tr>
boolean Stack<Tr>::IsEmpty()
{
    if (top==0) return true; else return false;
}
```

```
// prüfen, ob Stapel voll ist
template <class Tr>
boolean Stack<Tr>::IsFull()
{
    if (top==m) return true; else return false;
}
```

Datenstruktur Stack

1. Anwendungsbeispiel

- integer-Zahlen sollen auf den Stack gelegt werden und anschließend vom Stack geholt und wieder ausgegeben werden



Datenstruktur stack

2. Anwendungsbeispiel

Umwandlung von Infix in Postfix-Notation (UPN, RPN)

Infix-Notation: $x+y$

Postfix-Notation (UPN): x (enter) $y+$



INFIX $\Rightarrow$ POSTFIX Umwandlung

Stapel anfangs leer

Stapel enthält während des Algorithmus nur '(' oder Rechenoperatoren

Durchlaufe die Eingabeelemente von links nach rechts.

Fallunterscheidung:

1. '('
Klammer '(' auf Stapel legen
2. Zahl
Zahl ausgeben
3. Operator op
Alle Operatoren mit gleicher oder höherer Priorität vom Stapel ausgeben, bis '(' erreicht oder Stapel leer. Operator op auf Stapel legen.
4. ')'
Alle Operatoren vom Stapel bis zur ersten linken Klammer ausgeben.
Die erste linke Klammer aus dem Stack löschen.

Am Ende alle verbleibenden Operatoren vom Stapel ausgeben.

Auswertung eines Ausdrucks in Postfix-Notation

Stapel anfangs leer

Stapel enthält während des Algorithmus nur Zahlen

Durchlaufe die Eingabeelemente von links nach rechts.

Fallunterscheidung

1. Zahl: Lege die Zahl auf den Stapel
2. Operator op:
 - Wende den Operator op auf die beiden obersten Elemente des Stapels an
 - Ersetze diese beiden Zahlen durch das Ergebnis

Am Ende steht das Rechenergebnis als einziger Wert im Stapel

Datentyp Tabelle

- verfügbare Elementfunktionen:
 - generieren, entfernen von Tabellen
 - suchen, ausgeben, ersetzen, löschen von Einträgen in der Tabelle
 - Reorganisation der Tabelle
 - Sortieren der Einträge nach vorgegebenen Kriterien
 - Selektion nach vorgegebenen Kriterien
 - Typische Operation von ->Datenbanken
- Einträge (Elemente)
 - records eines Typs Tr

Datentyp Tabelle

Schnittstelle: öffentlich sichtbare Operationen:

```
template <class Tr>
class Table
{
public:
    Table(unsigned m); // kreiert leere Tabelle für m Records
    ~Table(); // gibt von der Tabelle belegten Speicherbereich frei
    void Reorg(unsigned m); // Tabelle um m zusätzliche Records erweitern
    boolean Append(Tr r); // fügt neuen Record am Ende ein
    boolean IsEmpty(); // true, wenn Tabelle leer, sonst false
    boolean IsFull(); // true, wenn Tabelle voll, sonst false
    unsigned Length(); // liefert aktuelle Länge der Tabelle
    boolean First(); // erster Record wird zum aktuellen Record
    boolean Last(); // letzter Record wird zum aktuellen Record
    boolean Next(); // der dem aktuellen Record folgende Record
                    // wird zum neuen aktuellen Record
    boolean Previous(); // der dem aktuellen Record vorausgehende
                       // Record wird zum neuen aktuellen Record
```

Datentyp Tabelle

Schnittstelle: öffentlich sichtbare Operationen:

```
boolean Insert(Tr &r); // fügt neuen Record vor dem aktuellen Record ein
boolean Delete();      // löscht aktuellen Record
boolean GetNode(Tr &r); // liefert aktuellen Record in Recordvariablen r
boolean SetNode(Tr &r); // ersetzt aktuellen Record durch Recordvariable r
```

Datentyp Tabelle

private Variablen und Methoden:

```
private:
    unsigned max, // maximale Länge der Tabelle
    n;           // aktuelle Länge der Tabelle
    int iterator; // Iterator, zeigt auf den aktuellen Record
    Tr* a;        // Zeiger auf array
};
```

Anwendungsbeispiel 1

- Tabelle des Recordtyps Node
 - Schlüsselfeld (unsigned int)
 - Infofeld (String)

Anwendungsbeispiel 1

- Tabelle t vom recordtyp Node mit maximal 100 records:

```
Table<Node> t(100);  
Node *r;  
int s=1;  
for (i=1; i<=10; i++)  
{  
    r=new Node;  
    r->SetKey(s++);  
    r->SetInfo („leer");  
    if (!t.Append(*r)) break;    // Abbruch, wenn Tabelle voll  
}
```

Anwendungsbeispiel 1

In Tabelle t im record mit Schlüssel x wird Inhalt des Feldes Info geändert:

```
Node r;
char str[50]="neue Information"; x=5;
if (t.First())      // Position auf Anfang
do
{
    t.GetNode(r);
    if (r.GetKey()==x)
    {
        r.SetInfo(str); // setze Info im record r
        t.SetNode(r);   // schreibe neues r in Tabelle t
    }
} while (t.Next())    // so lange bis Ende erreicht
```

Anwendungsbeispiel 2

- Klasse Student
 - Attribute:
 - Matrikelnummer = key = Primärschlüssel
 - Name
 - Studiengangsnummer (2=Inf)
 - Note
- Drucke Note von allen Studierenden im Studiengang Inf

Anwendungsbeispiel 2

```
Table <Student> m(50);
Student rm;
if (m.First())
{
    while (m.Next())
    {
        m.GetNode(rm);
        if (rm.Get_Studiengang() == 2)
        {
            cout << „Note von“ << rm.Get_Mat-Nummer();
            cout << „ist“ << rm.Get_Note() << endl;
        }
    }
};
```

STL – Standard Template Library

Implementiert Tabelle als Datenstruktur mit den Zugriffen wie angegeben:

Sequenzielle Container

Name	Klassenname	Beschreibung
Felder dynamischer Größe, Vector	<code>std::vector</code>	Einfügen und Löschen am Ende ist in $\mathcal{O}(1)$ und für anderen Elemente in $\mathcal{O}(n)$ möglich. Der Container unterstützt wahlfreien Zugriff (Random Access) in $\mathcal{O}(1)$.
Felder fester Größe	<code>std::tr1::array</code>	Erst seit <i>technical review 1</i> verfügbar.
doppelt verkettete Listen	<code>std::list</code>	Einfügen und Löschen ist in $\mathcal{O}(1)$ möglich. Wahlfreier Zugriff ist nicht möglich.
Warteschlangen	<code>std::queue</code>	Der Container unterstützt keine Iteratoren.
Warteschlangen mit zwei Enden	<code>std::deque</code>	Der Datentyp verhält sich wie der Vector, kann jedoch Elemente am Anfang und Ende in $\mathcal{O}(1)$ einfügen.
Warteschlangen mit Prioritäten	<code>std::priority_queue</code>	Die Struktur garantiert wie der Heap , dass immer das Element mit der höchsten Priorität am Beginn steht.
Stapel	<code>std::stack</code>	Der Container unterstützt keine Iteratoren.

Anwendung Datenbanken

Datenbankanwendungen bestehen aus Tabellen:

- Eindeutiger Primärschlüssel zur Identifikation nötig
 - Kann automatisch vergeben werden
 - Kann auch vom user vorgegeben werden (Buch-Signatur, Mat-Nummer,...)
- Beispiel Kursbelegung:
 - Tabelle Student
 - Enthält Objekte der Klasse Student (Primärschlüssel=Mat-Nr)
 - Tabelle Kurs
 - Enthält Objekte der Klasse Kurs (Primärschlüssel Kursnummer wird automatisch vergeben)
 - Beziehung Student belegt Kurs über Tabelle Belegt realisieren
 - Enthält Paare Mat-Nr.- Kursnummer

Anwendung Datenbanken



eindeutiges key-Feld = Primärschlüssel im Record nötig
Suche und Auswahl der Records nach unterschiedlichen Kriterien
Identifikation des Records über eindeutigen Primärschlüssel
-> LV Datenbanken

Record besteht aus Schlüssel und Daten



Implementierung Tabelle

- Tabelle als Array implementierbar:
 - feste zusammenhängende Random-Access-Struktur
 - direkter schneller Zugriff auf die Komponenten
 - Suchen und Sortieren im array leicht möglich

Implementierung Tabelle

Konstruktor:

```
// kreiert leere Tabelle für maximal m Records
template <class Tr>
Table<Tr>::Table(unsigned m) : max(m)
{
    a = new Tr[m+1];
    n = 0;
    iterator = -1;
}
```

Implementierung Tabelle

Destruktor

```
// gibt von der Tabelle belegten Speicherbereich frei
template <class Tr>
Table<Tr>::~~Table()
{
    delete [] a;
}
```

Implementierung Tabelle

Tabelle um m zusätzliche Records erweitern

```
template <class Tr>
void Table<Tr>::Reorg(unsigned m)
{
    Tr *b =new Tr[max+m+1];
    max+=m;
    for (int i=0; i<n; i++) b[i]=a[i];
    delete [] a;          // gibt den Speicher des Arrays a frei
    a=b;
}
```

Implementierung Tabelle

Einfügen eines neuen Records am Ende

```
template <class Tr>
boolean Table<Tr>::Append(Tr r)
{
    if (n<max)
    {
        a[n]=r;
        n++;
        return true;
    }
    else return false;
}
```

Implementierung Tabelle

// liefert aktuelle Länge der Tabelle

```
template <class Tr>
unsigned Table<Tr>::Length()
{
    return n;
}
```

Implementierung Tabelle

- Aktueller record
 - Operationen first, next, previous markieren einen Tabelleneintrag
 - Markierung wird in der Variablen iterator eingetragen
 - Elementfunktionen haben den Wert true, falls record gefunden, sonst false

Implementierung Tabelle

erster Record wird zum aktuellen Record

```
template <class Tr>
boolean Table<Tr>::First()
{
    if (n>0)
    {
        iterator=0;
        return true;
    }
    iterator=-1;
    return false;
}
```

Implementierung Tabelle

// nächster Record wird zum aktuellen Record

```
template <class Tr>
boolean Table<Tr>::Next()
{
    if (n>0 && iterator>=0 && iterator<n-1)
    {
        iterator++;
        return true;
    }
    iterator=-1;
    return false;
}
```

Implementierung Tabelle

// vorhergehender Record wird zum aktuellen Record

```
template <class Tr>
boolean Table<Tr>::Previous()
{
    if ((iterator>0) && (n>0))
    {
        iterator--;
        return true;
    }
    iterator=-1;
    return false;
}
```

Implementierung Tabelle

true, wenn Tabelle leer, sonst false

```
template <class Tr>
bool Table<Tr>::IsEmpty()
{
    if (n==0) return true;
    else return false ;
}
```

true, wenn Tabelle voll, sonst false

```
template <class Tr>
bool Table<Tr>::IsFull()
{
    if (n==max) return true;
    else return false;
}
```

Implementierung Tabelle

- einfügen eines records vor dem aktuellen record
 - bestehende Tabelle muß ab der iterator-Position um einen Platz nach hinten verschoben werden
- kann nur für nicht leere oder nicht volle Tabelle durchgeführt werden
- es muß ein aktueller record bestimmt sein

Implementierung Tabelle

fügt neuen Record vor dem aktuellen Record ein

```
template <class Tr>
boolean Table<Tr>::Insert(Tr &r)
{
    if (iterator>=0 && n<max)
    {
        for (int i=n-1; i>=iterator; i--)
            a[i+1]=a[i]; // umspeichern einschließlich Iterator-Index
        a[iterator]=r;
        n++; // Anzahl Records aktualisieren
        iterator++; // Iterator zeigt auf den alten Record
        return true;
    }
    else return false;
}
```

Implementierung Tabelle

löscht aktuellen Record

```
template <class Tr>
boolean Table<Tr>::Delete()
{
    if (iterator<0) return false;
    for (int i=iterator; i<n-1; i++) a[i]=a[i+1];
    n--;
    iterator=-1;
    return true;
}
```

Implementierung Tabelle

liefert aktuellen Record

```
template <class Tr>
boolean Table<Tr>::GetNode(Tr &r)
{
    if (iterator>=0)
    {
        r=a[iterator];
        return true;
    }
    return false;
}
```

Implementierung Tabelle

ersetzt aktuellen Record durch Record r

```
template <class Tr>
bool Table<Tr>::SetNode(Tr &r)
{
    if (iterator>=0)
    {
        a[iterator]=r;
        return true;
    }
    return false;
}
```

Komplexität

Reorg hat Komplexität $O(n)$

Einfügen und Löschen hat im ungünstigsten Fall Komplexität $O(n)$

alle anderen Routinen $O(1)$

Suchen:

- lineare Suche $O(n)$
- Binäre Suche $O(\log n)$, aber Sortierung nötig!

Damit die Suche schnell ist, ist die Sortierung der Tabelle z.B. nach dem Schlüsselfeld nötig!